



North South University
Department of Electrical & Computer Engineering
PROJECT REPORT (Spring 2024)

Course Name: **CSE332L**

Section: **07**

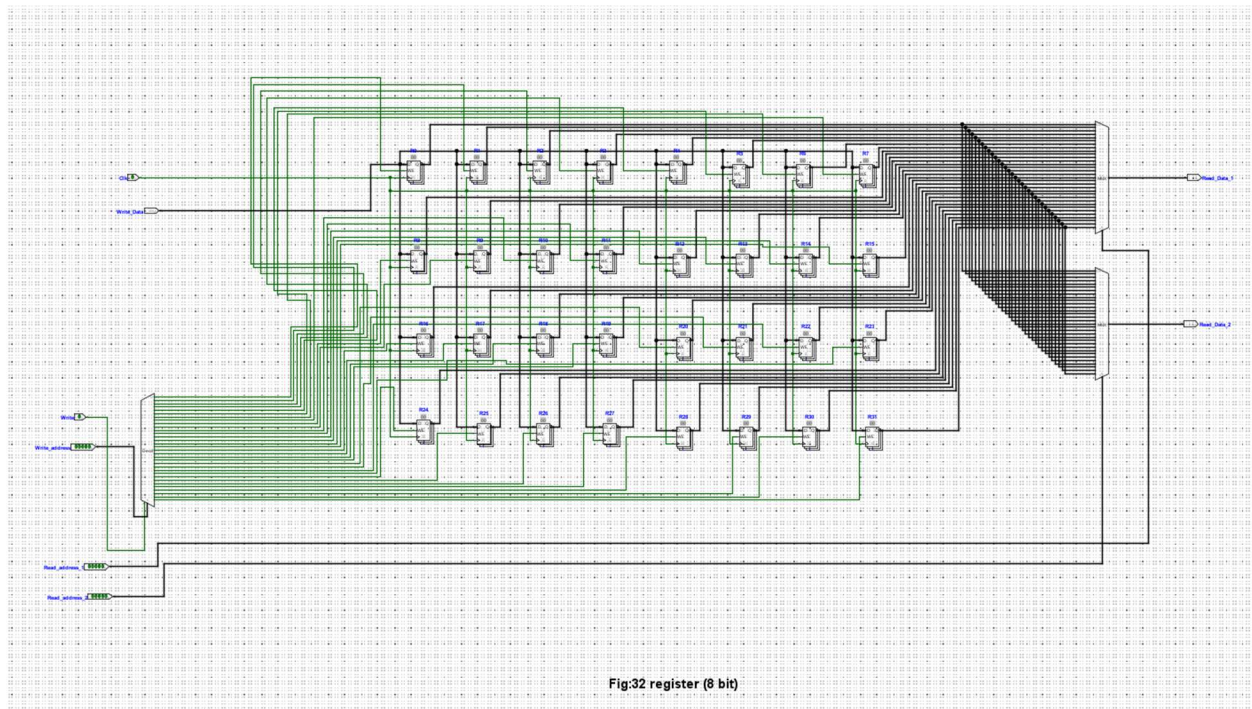
Topic: **Store first 10 numbers of the series $4+8+12+16+20+24+\dots+n$, in memory and their sum on SR1 using loop and bits for the architecture 22 and Bonus part (array data sorting)**

Made and Submitted by Makki Ammer Sakib (2231687642)	Submitted to Md. Wardun Islam Noon (Lab Instructor)
Remarks:	Score

Discussion and Explanation:

Bit Selection and Register File:

Firstly, I designed a register file where the number of registers is 32 and the register data bit is 8. Because I have to design 22-bit processor. For 32 registers, rs (source register), rt (target register), rd (register destination) is 5 bits each and for 8 bit register shamt 3 bits and Opcode 4 bits. This is bit distribution for r-type instruction. For i-type opcode 4 bits, rs (source register), rd (register destination) is 5 bits each and immediate data 8 bits. For j-type 4 bits opcode and other 18 bits needed for jump operation.



ALU Table and File:

ALU OPCODE	OPERATOR
000	ADDER
001	SUBTRACTOR
010	INCREMENT
011	LEFT SHIFTER
100	RIGHT SHIFTER
101	XOR
110	COMPARATOR (ONLY 1 WHEN A>B)
111	NO OPERATOR

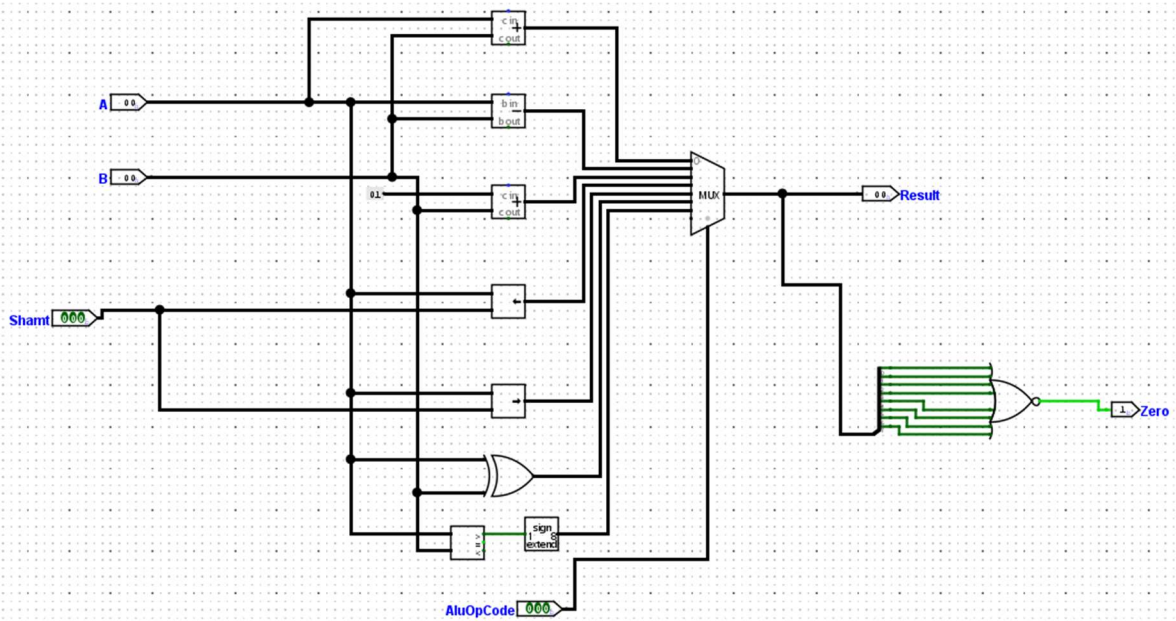


Fig: 8 bit ALU

Datapath File:

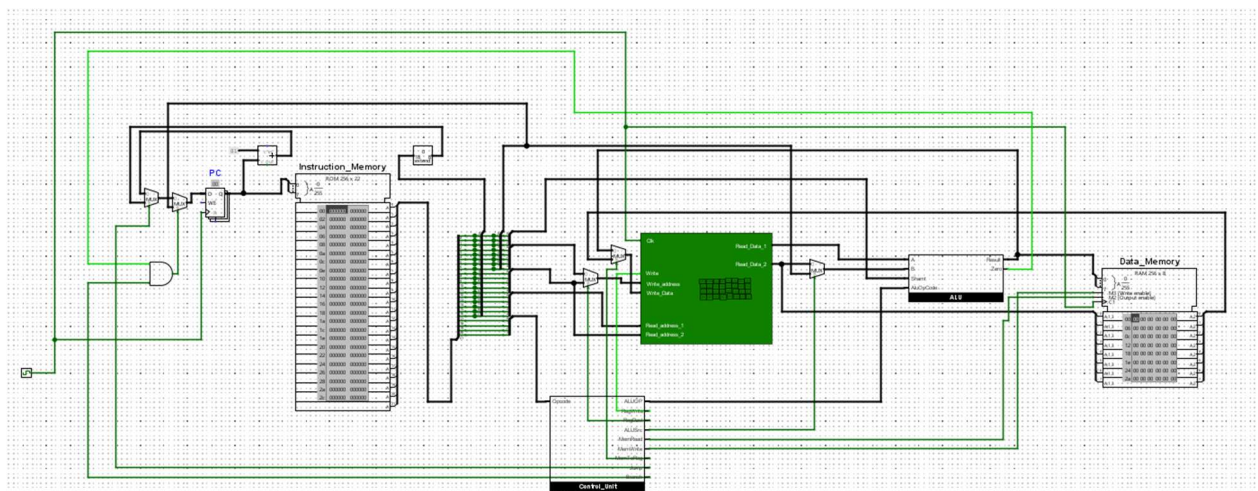
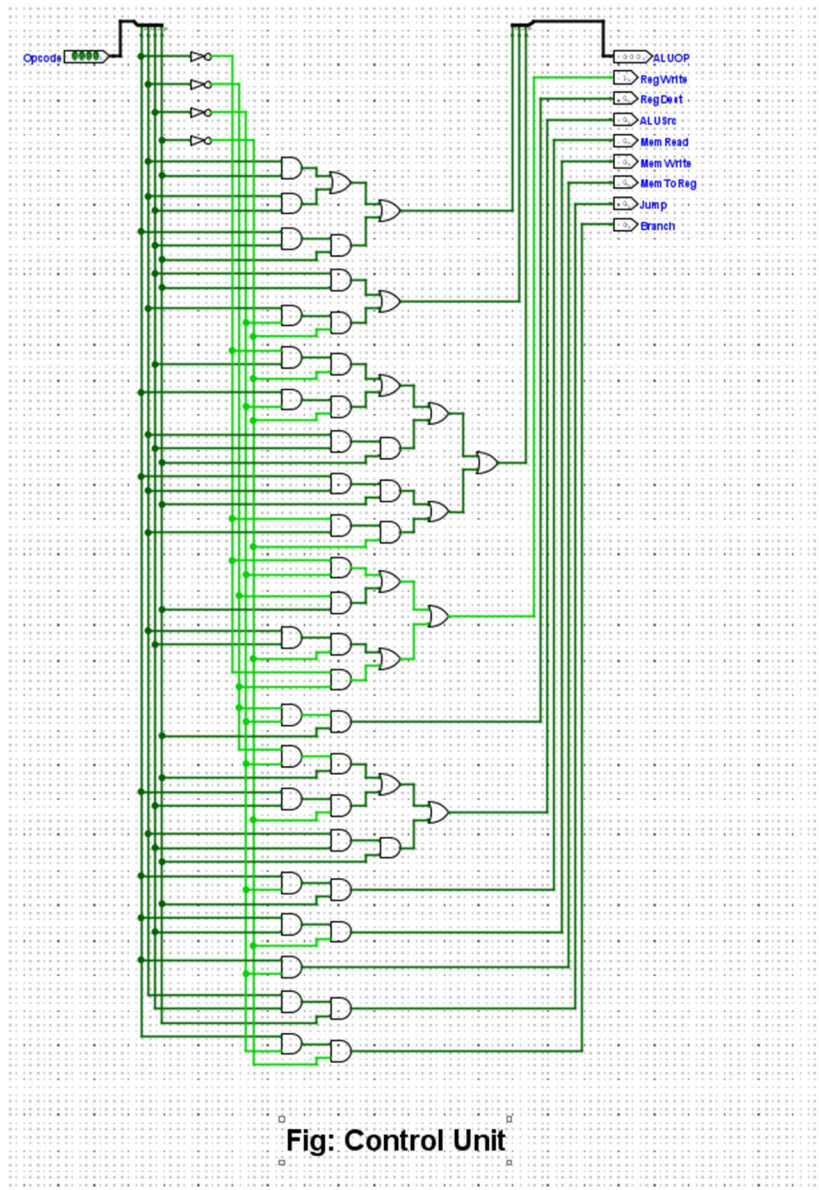


Fig: Datapath

Control Unit Table and File:

Operation	OPCODE	ALUOP	RegWrite	RegDest	ALUSrc	MemRead	MemWrite	MemToREg	Jump	Branch
ADD	0000	000	1	0	0	0	0	0	0	0
ADDI	0001	000	1	1	1	0	0	0	0	0
SUB	0010	001	1	0	0	0	0	0	0	0
INC	0011	010	1	0	0	0	0	0	0	0
SHIFLEFT	0100	011	1	0	0	0	0	0	0	0
SHIFTRIGHT	0101	100	1	0	0	0	0	0	0	0
XOR	0110	101	1	0	0	0	0	0	0	0
JUMP	0111	xxx	0	x	x	0	0	x	1	0
BRANCH	1000	001	0	x	0	0	0	x	0	1
LOAD	1001	000	1	1	1	1	0	1	0	0
STORE	1010	000	0	x	1	0	1	x	0	0
>	1011	110	1	0	0	0	0	0	0	0
NO OPERATION	1100	xxx	x	x	x	x	x	x	x	x
NO OPERATION	1101	xxx	x	x	x	x	x	x	x	x
NO OPERATION	1110	xxx	x	x	x	x	x	x	x	x
NO OPERATION	1111	xxx	x	x	x	x	x	x	x	x



Instruction:

- Store first 10 numbers of the series $4+8+12+16+20+24+\dots+n$, in memory and their sum on \$R1 using loop.

C code:

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```

int main()
{
    int i=0;
    int a=4;
    int add=0;
    int n=10;
    while(i!=n)
    {
        add=add+a;
        printf("%d ",a);
        a=a+4;
        i=i+1;
    }
    printf("%d ",add);

    return 0;
}

```

Compiled code/ Instruction format for 22-bit processor's Datapath:

0 int i=0; (i-type) (Initializing i)

ADDI R0 R1 0

0001 00000 00001 000000000

Hex-

40100

1 int a=4; (i-type) (Initializing the first number of the series which is 4)

ADDI R0 R2 4

0001 00000 00010 00000100

Hex-

40204

2 int add=0; (i-type) (This is the variable where add the number of the series until breaks the loop)

ADDI R0 R3 0

0001 00000 00011 00000000

Hex-

40300

3 int n=10; (i-type) (this is the condition checker, when it equals to i then loop breaks)

ADDI R0 R4 10

0001 00000 00100 00001010

Hex-

4040A

4 R5=5 (i-type) (this is the offset part of the store address for first 10 numbers of that series)

ADDI R0 R5 5

0001 00000 00101 00000101

Hex-

40505

5 R6=1 (i-type) (this is the offset part for the store address of the sum of the series)

ADDI R0 R6 1

0001 00000 00110 00000001

Hex-

40601

6 while(i!=n) { (Branch) ((There loop starts to store the number of this series and store the sum of this series)

BEQ R1 R4 14

1000 00001 00100 00001110

Hex-

20240E

7 add=add+a;(r-type) (every numbers of this series adding)

ADD R3 R2 R3 0

0000 00011 00010 00011 000

Hex-

6218

8 printf("%d ",a); (Store) address for store 10 numbers of this series in data memory address 6 to 15(i-type) (in 4th line offset 5 and there add 1 which is the first address to store the value of a)

1010 00101 00010 00000001

Hex-

28A201

9 R5+1 address

ADDI R5 R5 1 (need to increment the address. to store the values of a sequentially)

0001 00101 00101 00000001

Hex-

4A501

10 a=a+4; (i-type) (adding 4. because the pattern of the series)

ADDI R2 R2 4

0001 00010 00010 00000100

Hex-

44204

11 i=i+1; (i-type) (increment of i)

ADDI R1 R1 1

0001 00001 00001 00000001

Hex-

42101

12 printf("%d ",add); (Store) (in 5th line the offset was 1 and now add with 15. so in data memory address 16 store the sum.

STORE R6 R3 15

1010 00110 00011 00001111

Hex-

28C30F

13 } this is the line to continue the loop. every time loop check the condition and every statement of this loop works and again jump to 6 to check the condition again.

Jmp 6 (j-type)

0111 0000000000000000110

Hex-

1C0006

14 Stop

After run this instruction in the processor's Datapath my output:

00	00 00 00 00	00 00 04 08	0c 10 14 18	1c 20 24 28
10	dc 00 00 00	00 00 00 00	00 00 00 00	00 00 00 00
20	00 00 00 00	00 00 00 00	00 00 00 00	00 00 00 00
30	00 00 00 00	00 00 00 00	00 00 00 00	00 00 00 00
40	00 00 00 00	00 00 00 00	00 00 00 00	00 00 00 00
50	00 00 00 00	00 00 00 00	00 00 00 00	00 00 00 00
60	00 00 00 00	00 00 00 00	00 00 00 00	00 00 00 00
70	00 00 00 00	00 00 00 00	00 00 00 00	00 00 00 00
80	00 00 00 00	00 00 00 00	00 00 00 00	00 00 00 00
90	00 00 00 00	00 00 00 00	00 00 00 00	00 00 00 00
a0	00 00 00 00	00 00 00 00	00 00 00 00	00 00 00 00
b0	00 00 00 00	00 00 00 00	00 00 00 00	00 00 00 00
c0	00 00 00 00	00 00 00 00	00 00 00 00	00 00 00 00
d0	00 00 00 00	00 00 00 00	00 00 00 00	00 00 00 00
e0	00 00 00 00	00 00 00 00	00 00 00 00	00 00 00 00
f0	00 00 00 00	00 00 00 00	00 00 00 00	00 00 00 00

All outputs are in hexadecimal format. 4,8,,c,10,14,18,1c,20,24,28 are in decimal 4,8,12,16,20,24,28,32,36,40. dc is the sum of this 10 numbers. which is 220 in decimal.

Bonus part:

- In addition to the core project tasks, you have the opportunity to earn a bonus by implementing a sorting algorithm of your choice in assembly language. This sorting algorithm should be capable of sorting an array of data. You are encouraged to choose and implement an algorithm that aligns with the capabilities of your processor. The sorting algorithm should be well-documented and tested as part of your project submission.*

For this bonus part, I have to take an array randomly. I took Array[22] = {18,3,32,17,1,7}. 22nd indicates the data memory address of the first element of the array. Then I convert this number into a hexadecimal number. 18,3,32,17,1,7 in hexadecimal: 12,3,20,11,1,7.

There are 6 elements in this array. Firstly, stored 12 in 22 data memory address of the memory and sequentially stored other elements. So, 7 are stored in 27th data memory address.

```

00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
10 00 00 00 00 00 00 12 03 20 11 01 07 00 00 00 00
20 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
30 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
40 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
50 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
60 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
70 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
80 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
90 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
a0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
b0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
c0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
d0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
e0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
f0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00

```

Then I compiled a c code to sort this array and write an instruction format for my processor.

C Code:

```
#include <stdio.h>
```

```
int main() {
```

```
    int array[5]={12,3,32,17,1,7}, y=5,m,n,t=0;    /* here 5 means size and in
processor it is 22 because I store first element of the array in 22 address */
```

```
    while (t != y) {
```

```
        int j=0;
```

```
        while (j != y-t) {
```

```
            int m=array[j];
```

```
            int n=array[j+1];
```

```
            if (m>n) {
```

```
                array[j] = n;
```

```
                array[j+1] = m;
```

```
            }
```

```

        j++;
    }
    i++;
}

```

Compiled code/ Instruction format for 22-bit processor's Datapath:

14 int t=0; (i-type) (Initializing t)

ADDI R0 R9 0

0001 00000 01001 00000000

Hex-

40900

15 int y=5; (i-type)

ADDI R0 R10 5

0001 00000 01010 00000101

Hex-

40A05

16

int k=0; (i-type) (Initializing k)

ADDI R0 R19 0

0001 00000 10011 00000000

Hex-

41300

17 while (t != y) {

BEQ R9 R10 33

1000 01001 01010 00100001

Hex-

212A21

18 (offset)

int R7=5; (i-type)

ADDI R0 R7 5

0001 00000 00111 00000101

Hex-

40705

19 inner loop condition checker

int j=0; (i-type) (Initializing j)

ADDI R0 R20 0

0001 00000 10100 00000000

Hex-

41400

20 y-t

SUB R10 R9 R25 shamt

0010 01010 01001 11001 000

Hex-

949C8

```
# 21 while (j != y-t) {  
# BEQ R20 R25 31  
# 1000 10100 11001 00011111  
# Hex-  
22991F
```

```
# 22 int m=array[j];  
# LW R7 R13 17 (offset 5 and 17 means 22 which is the address of the first  
# element of the array load temporarily in 13th register))  
# 1001 00111 01101 00010001  
# Hex-  
24ED11
```

```
# 23 int n=array[j+1]; (offset 5 and 18 means 23 which is the address of the second  
# element of the array load temporarily in 14th register)  
# LW R7 R14 18  
# 1001 00111 01110 00010010  
# Hex-  
24EE12
```

```
# 24      if (m>n) {  
# CMP R13 R14 R15 000  
# 1011 01101 01110 01111 000  
# Hex-  
2DAE78
```

25 R15 is 1 when m or R13 is greater than n or R14 register. If 1 then go to the 26th line which is inside of if parenthesis. There is compare R15 with R19. Which is initialized in 16th line as k=0. If R15 0 then skip 26th line and 27th line.

```
# BEQ R15 R19 28  
# 1000 01111 10011 00011100  
# Hex-  
21F31C
```

```
# 26  array[j] = n;  
# SW R7 R14 17  
# 1010 00111 01110 00010001  
# Hex-  
28EE11
```

```
# 27  array[j+1] = m;  
# SW R7 R13 18  
# 1010 00111 01101 00010010
```

Hex-
28ED12

28 address increment

R7+1

ADDI R7 R7 1

0001 00111 00111 00000001

Hex-
4E701

29 inner loop increment

j++

ADDI R20 R20 1

0001 10100 10100 00000001

Hex-
69401

30 } loop

Jmp 20(j-type)

0111 000000000000010100

Hex-
1C0014

31 t++ (i-type) (increment of t) outer loop increment

ADDI R9 R9 1

0001 01001 01001 00000001

Hex-

52901

32 } loop

Jmp 17(j-type)

0111 000000000000010001

Hex-

1C0011

After run this instruction in the processor's Datapath my output:

00	00 00 00 00	00 00 00 00	00 00 00 00	00 00 00 00
10	00 00 00 00	00 00 01 03	07 11 12 20	00 00 00 00
20	00 00 00 00	00 00 00 00	00 00 00 00	00 00 00 00
30	00 00 00 00	00 00 00 00	00 00 00 00	00 00 00 00
40	00 00 00 00	00 00 00 00	00 00 00 00	00 00 00 00
50	00 00 00 00	00 00 00 00	00 00 00 00	00 00 00 00
60	00 00 00 00	00 00 00 00	00 00 00 00	00 00 00 00
70	00 00 00 00	00 00 00 00	00 00 00 00	00 00 00 00
80	00 00 00 00	00 00 00 00	00 00 00 00	00 00 00 00
90	00 00 00 00	00 00 00 00	00 00 00 00	00 00 00 00
a0	00 00 00 00	00 00 00 00	00 00 00 00	00 00 00 00
b0	00 00 00 00	00 00 00 00	00 00 00 00	00 00 00 00
c0	00 00 00 00	00 00 00 00	00 00 00 00	00 00 00 00
d0	00 00 00 00	00 00 00 00	00 00 00 00	00 00 00 00
e0	00 00 00 00	00 00 00 00	00 00 00 00	00 00 00 00
f0	00 00 00 00	00 00 00 00	00 00 00 00	00 00 00 00

All outputs are in hexadecimal format and my array data sort in increasing order. 1,3,7,11,12,20 in decimal 1,3,7,17,18,32. So array data was 18,3,32,17,1,7. After sorting 1,3,7,17,18,32.